МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ЛАБОРАТОРНОЙ РАБОТЕ

К1

Параллельная сортировка больших массивов

III. Комбинаторные вычисления • Базовый уровень сложности

Среда: Windows 10 · MSVC C++17 · OpenMP · NVIDIA CUDA · Python (pandas, matplotlib).

Предпосылка: диагностическая ЛР № 0 выполнена; компиляторы и CUDA проверены. При отсутствии совместимой NVIDIA на личном ноутбуке GPU-эксперименты выполняются на кафедральной машине.

Суть исследования

Сравнить последовательную, многопоточную CPU- и массово-параллельную GPU-реализации одного алгоритма сортировки. Основной алгоритм — битонная сортировка (Bitonic Sort), основанная на сети попарных сравнений. В качестве практического ориентира отдельно использовать std::sort. Изучить влияние размера массива, входного распределения, числа потоков, запусков CUDA-ядер и операций с памятью.

Прикладное значение

Сортировка используется в аналитических платформах и СУБД (ORDER BY, группировка, сортировочные соединения), подготовке данных, ранжировании, поиске дубликатов и вычислении порядковых статистик (медианы, квантили). В финансовых системах это может быть упорядочение транзакций по времени, сумме или приоритету обработки. Практический вывод лабораторной: не всякая сортировка одинаково эффективна на CPU и GPU; важно учитывать структуру алгоритма и стоимость перемещения данных.

Что надо сдать?

- Исходники трёх реализаций: последовательная bitonic CPU, OpenMP bitonic CPU, CUDA bitonic GPU; отдельное сравнение со std::sort и README с командами сборки.
- Тесты корректности (включая дубликаты, крайние случаи и различные исходные порядки) с автоматическим сравнением с эталонной отсортированной последовательностью.
- CSV с измерениями по каждому запуску, таблицы и графики, краткий аналитический отчёт с выводами.
- Защиту: объяснение сети сравнений, безопасности попарных обменов, барьеров между стадиями и различий между ускорением алгоритма и практической производительностью.

Все массивы в обязательной части имеют тип uint32_t, сортировка производится по возрастанию.

1. Планируемые результаты и границы задания

1.1. Студент должен уметь

- Объяснить, что такое сортировка сравнением, сортировочная сеть, битонная последовательность и стадия compare-and-swap.
- Реализовать корректную битонную сортировку для длины N = 2^m и обосновать требование к размеру массива.
- Организовать независимые попарные обмены на CPU средствами OpenMP и на GPU средствами CUDA.
- Различать барьер синхронизации OpenMP, синхронизацию потоков внутри CUDA-блока и границу между запусками CUDA-ядер.
- Сопоставить время сортировки, пропускную способность, потребление памяти и ускорение; отделить время GPU-вычислений от времени всей GPU-операции.
- Оценить, когда учебный параллельный алгоритм уступает хорошо оптимизированной стандартной сортировке.

1.2. Что намеренно не требуется

Не требуется реализация промышленной высокопроизводительной GPU-сортировки, произвольных пользовательских компараторов и оптимизация через shared memory. Они оставлены для дополнительных исследований. Базовый уровень должен быть доступен на RTX 3060 с 12 ГБ VRAM и типичных студенческих ноутбуках, при разумном выборе N.

2. Теория: что и как мы сортируем

2.1. Сортировка сравнением и её сложность

Для произвольных входных массивов сортировка сравнением имеет нижнюю оценку $\Omega(N \log_2 N)$ сравнений в худшем случае. Стандартный std::sort обеспечивает асимптотическую сложность O(N log N) сравнений и хорошо оптимизирован для универсальных CPU; точные детали реализации зависят от стандартной библиотеки. Мы используем его как эталон правильного результата и отдельный показатель практического времени.

Битонная сортировка относится к сортировочным сетям: последовательность пар сравниваемых индексов задана размером массива и не зависит от значений элементов. Это делает алгоритм удобным для распараллеливания и демонстрации синхронизации.

2.2. Битонная последовательность и compare-and-swap

Битонной называют последовательность, которую можно представить как сначала неубывающую, затем невозрастающую (с учётом циклического сдвига). Битонная сеть сортировки последовательно формирует и сливает такие последовательности. Элементарная операция compare-and-swap сравнивает два элемента и меняет их местами, если их порядок не соответствует направлению стадии.

При N = 2^m алгоритм имеет m(m + 1)/2 стадий попарных сравнений, а суммарная работа — $O(N \log^2 N)$. На одной стадии пары индексов не пересекаются, поэтому обмены можно выполнять параллельно. После стадии необходима синхронизация перед следующей стадией.

2.3. Правило выбора пары и направления

Обозначим i — индекс элемента; k — размер обрабатываемой битонной последовательности; j — расстояние между сравниваемыми индексами. Партнёр вычисляется как partner = i XOR j. Обмен выполняется только для i < partner, чтобы каждая пара обрабатывалась ровно один раз. Направление определяется битом i AND k: при (i AND k) = 0 сравниваем по возрастанию, иначе — по убыванию.

```
for (k = 2; k <= N; k *= 2)
    for (j = k / 2; j > 0; j /= 2)
        parallel compare-and-swap(i, i XOR j) for all i < (i XOR j)
```

Схема показывает структуру, а не готовую программу. Между последовательными значениями (k, j) должны завершиться все обмены текущей стадии.

2.4. Первые стадии сети при N = 8

k	j	Независимые пары на стадии	Число пар
2	1	(0,1), (2,3), (4,5), (6,7)	4
4	2	(0,2), (1,3), (4,6), (5,7)	4
4	1	(0,1), (2,3), (4,5), (6,7)	4
8	4	(0,4), (1,5), (2,6), (3,7)	4
8	2	(0,2), (1,3), (4,6), (5,7)	4
8	1	(0,1), (2,3), (4,5), (6,7)	4

Важно: одинаковые пары могут встречаться на разных стадиях; при этом направление обмена определяется k. Для массива из восьми элементов требуется шесть стадий. Эта таблица удобна для пошаговой отладки.

2.5. Почему битонная сортировка — учебный, а не универсальный выбор?

Её преимущество — фиксированная схема обменов и наглядный параллелизм. Ограничение — больше работы, чем у O(N log N)-алгоритмов, и множество синхронизируемых стадий. Для N = 2^20 потребуется 210 стадий; простая CUDA-реализация выполняет отдельный запуск ядра для каждой из них. Поэтому GPU-версия вовсе не обязана превзойти std::sort на CPU.

3. Подготовка проекта и исходных данных

3.1. Рекомендуемая структура

```
K1/
  src/
    sort_cpu.cpp       // serial bitonic, OpenMP bitonic, std::sort
    sort_cuda.cu       // GPU bitonic
  scripts/
    plots.py           // графики и статистика
  data/
    measurements.csv   // каждое повторение — отдельная строка
  README.md
  report.docx          // или report.pdf
```

3.2. Генерация и воспроизводимость

Для каждой пары (N, тип распределения) генерируйте один исходный массив A, сохраняйте seed и подавайте его независимые копии всем сравниваемым реализациям. Не допускается, чтобы одна реализация получала уже отсортированный массив после предыдущей.

Набор данных	Как сформировать	Что выявляет
Uniform	Псевдослучайные uint32_t в широком диапазоне	Типовая смешанная последовательность
Sorted	Уже отсортированный исходный массив	Наличие / отсутствие адаптивности алгоритма
Reverse	Элементы в убывающем порядке	Чувствительность к обратному порядку
FewUnique	Случайные значения из 16 различных ключей	Множество дубликатов
NearlySorted	Отсортированный массив с 1% случайных перестановок	Частично подготовленные данные

Для почти отсортированного массива фиксируйте способ выбора пар перестановок и общий seed. Рекомендуемый генератор C++: std::mt19937_64. На каждом тесте заранее вычисляйте эталон через std::sort над копией A.

3.3. Допустимые размеры

В обязательной части используйте степени двойки: N = 2^10, 2^14, 2^17, 2^20 (при необходимости добавьте 2^12 и 2^16). Для отладки сначала примените N = 2, 4, 8, 16. На ноутбуке, где максимальный размер выполняется слишком долго, можно ограничить N значением 2^17 и описать причину.

4. CPU: последовательная реализация и OpenMP

4.1. Последовательная версия

Реализуйте два вложенных цикла по k и j, а затем проход по всем индексам i. Перемена мест элементов происходит только для i < partner. Не используйте std::sort внутри собственной реализации bitonic.

```
for (std::size_t k = 2; k <= a.size(); k <<= 1) {
    for (std::size_t j = k >> 1; j > 0; j >>= 1) {
        for (std::size_t i = 0; i < a.size(); ++i) {
            const std::size_t partner = i ^ j;
            if (i >= partner) continue;
            const bool ascending = (i & k) == 0;
            // TODO: compare-and-swap для ascending / descending
        }
    }
}
```

Перед сортировкой проверяйте N > 0 и (N & (N − 1)) == 0. Для изложения используется N = 2^m; обработку произвольной длины через дополнение до степени двойки оставляем дополнительным заданием.

4.2. Параллельная версия OpenMP

Параллелизм появляется внутри каждой стадии (k, j): разные пары обрабатываются независимо. Используйте #pragma omp parallel for schedule(static) для распределения индексов. Внешние циклы по k и j остаются последовательными. По завершении omp for необходим барьер, чтобы следующая стадия не читала незавершённые обмены предыдущей.

```
#pragma omp parallel for schedule(static)
for (std::int64_t i = 0; i < static_cast<std::int64_t>(N); ++i) {
    const std::size_t pos = static_cast<std::size_t>(i);
    const std::size_t partner = pos ^ j;
    if (pos < partner) {
        const bool ascending = (pos & k) == 0;
        // TODO: compare-and-swap
    }
}
```

Данный фрагмент располагается внутри циклов k и j. В базовом варианте команда parallel for создаёт синхронизацию по окончании цикла (без nowait). Замеряйте весь алгоритм, а не отдельную стадию. Для MSVC базовая опция /openmp достаточна; специфические возможности новых версий OpenMP здесь не требуются.

4.3. Сборка из терминала

```
cl /std:c++17 /O2 /EHsc /openmp src\sort_cpu.cpp /Fe:sort_cpu.exe
sort_cpu.exe --mode serial --n 65536 --input uniform --seed 2026
sort_cpu.exe --mode omp --n 65536 --threads 4 --input uniform --seed 2026
sort_cpu.exe --mode std --n 65536 --input uniform --seed 2026
```

Аргументы командной строки — предлагаемый интерфейс, их разбор студент реализует самостоятельно. Команды выполняются в x64 Native Tools Command Prompt. Для честного эксперимента все режимы собираются в Release с сопоставимыми параметрами оптимизации.

4.4. Эксперимент с числом потоков

Проверьте 1, 2, 4, 6, 8, 12 потоков при наличии такого числа логических процессоров. Для кафедрального i5-11400F (6 физических ядер / 12 логических потоков) не следует заранее считать 12 потоков оптимальными: барьеры на каждой стадии и накладные расходы могут сделать меньший p быстрее. Сравните как минимум два размера N.

5. GPU: реализация битонной сети на CUDA

5.1. Разбиение работы

CUDA-ядро отвечает за одну стадию (k, j). GPU-поток с глобальным индексом i вычисляет партнёра i XOR j и при i < partner выполняет единственный compare-and-swap. Запускайте сетку блоков, покрывающую все N индексов. Лишние GPU-потоки выходят по условию i >= N.

```
__global__ void bitonic_step(std::uint32_t* a,
                             unsigned N, unsigned j, unsigned k) {
    const unsigned i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;
    const unsigned partner = i ^ j;
    if (i < partner) {
        const bool ascending = (i & k) == 0;
        // TODO: безопасный compare-and-swap(a[i], a[partner])
    }
}
```

Корректность основана на том, что в одной стадии пары индексов не пересекаются. Для базовой реализации данные находятся в глобальной памяти GPU. Shared memory используется только в факультативном исследовании.

5.2. Синхронизация: принципиальная ловушка

Вызов __syncthreads() синхронизирует потоки только внутри ОДНОГО CUDA-блока, а не всей сетки блоков. Поэтому нельзя помещать всю сеть с несколькими глобальными стадиями в одно обычное ядро и ожидать, что __syncthreads() создаст общий барьер. В базовом решении каждая стадия — отдельный запуск CUDA-ядра; запуски в одном потоке CUDA (stream) выполняются последовательно.

После каждого запуска проверяйте немедленные ошибки запуска; перед считыванием результата обеспечьте завершение GPU-работы и проверьте отложенные ошибки выполнения. Не выполняйте cudaDeviceSynchronize() после каждой стадии без причины: отдельная синхронизация с CPU увеличит накладные расходы.

5.3. Пример сборки

```
nvcc -O3 -std=c++17 -arch=sm_86 src\sort_cuda.cu -o sort_cuda.exe
sort_cuda.exe --n 65536 --input uniform --seed 2026 --block 256
```

sm_86 подходит RTX 3060 кафедрального компьютера. Для других GPU архитектуру нужно выбрать по Compute Capability и версии установленного CUDA Toolkit. nvcc под Windows использует совместимый компилятор MSVC.

5.4. Что именно измеряем

Метрика	Границы измерения
T_kernel	Время всех CUDA-стадий (включая необходимые запуски ядер), по CUDA Events и завершению работы в нужном stream.
T_transfer	Копирование исходного массива CPU → GPU и результата GPU → CPU (можно отдельно два направления).
T_total	Полный цикл: копирование входа, GPU-сортировка, копирование результата и необходимая синхронизация. Границы измерения фиксируются в отчёте.
T_alloc (отдельно)	При желании — выделение/освобождение буферов. Не смешивать замеры с T_total, если это не указано.

Для каждого повтора сортируйте свежую копию одинакового исходного массива: нельзя измерять второй проход на данных, оставшихся отсортированными после первого. Подготовку генератора исходных данных отделяйте от времени сортировки, если измеряется именно алгоритм.

6. Экспериментальная программа

6.1. Минимальный план

Переменная	Обязательные значения / условия
Размер N	2^10, 2^14, 2^17, 2^20 (по возможности)
Распределение	Uniform, Sorted, Reverse, FewUnique, NearlySorted
CPU режим	Serial bitonic, OpenMP bitonic, std::sort
Число потоков	1, 2, 4, 6, 8, 12 (ограничить аппаратно)
GPU	CUDA bitonic, блоки по 128 и 256 потоков; 512 — по возможности
Повторения	1–2 прогрева; не менее 10 замеров каждого выбранного случая
Правильность	Сравнить весь выходной массив с std::sort для того же входа

Чтобы объём эксперимента не стал чрезмерным, используйте полный набор распределений для одного среднего N (например, 2^17), а для исследования масштабируемости по N — Uniform. Измерения по потокам и размерам GPU-блока проводите на 2–3 выбранных N. Таким образом получается содержательный, но выполнимый план.

6.2. Описание CSV

```
hardware_id,algorithm,device,input_type,n,threads,block_size,
seed,repeat,time_ms,kernel_ms,transfer_ms,correct
```

Каждая строка CSV — один измеренный запуск. Не сохраняйте только усреднённые значения. Если поле не применимо, оставляйте его пустым. hardware_id должен однозначно связываться с паспортом системы из ЛР № 0; не требуется указывать персональные данные.

6.3. Показатели

Для сравнения вариантов одной битонной сети: S_CPU(p) = T_bitonic_serial / T_bitonic_omp(p); S_GPU = T_bitonic_serial / T_gpu_total. Для оценки практической конкурентоспособности дополнительно вычислите R_std = T_std_sort / T_gpu_total: если R_std < 1, стандартная CPU-сортировка быстрее GPU-решения.

Пропускная способность: Q = N / T (элементов/с), при указании времени T в секундах. Эффективность CPU: E(p) = S_CPU(p) / p. Число CUDA-потоков не следует подставлять вместо p в эту формулу: это другой способ организации вычислений.

6.4. Обязательные графики

1. Время сортировки от N (логарифмические оси при необходимости): serial bitonic, OpenMP bitonic, CUDA T_total и std::sort.
2. CPU-ускорение S_CPU(p) от числа потоков для N = 2^14 и N = 2^17 (или других обоснованных N).
3. CUDA: T_kernel и T_total от N; отдельно показать эффект размера CUDA-блока.
4. Влияние исходного порядка (пять распределений) на скорость std::sort и битонных реализаций при одном фиксированном N.
5. Пропускная способность N/T от N для всех основных реализаций.

6.5. Вопросы к экспериментальным выводам

- Почему более параллельная битонная реализация может уступать std::sort на CPU?
- Откуда появляется минимум времени при количестве потоков меньше числа логических процессоров?
- Как влияет исходный порядок на сортировочную сеть и на std::sort?
- Почему T_kernel на GPU не равен T_total и какой показатель важнее для прикладного заказчика?
- Сколько запусков CUDA-ядер выполняется при N = 2^10, 2^17 и 2^20? Как их количество влияет на результат?

7. Корректность и диагностика ошибок

Тест	Что должно происходить
N = 0	Корректное сообщение о некорректном размере; без обращения к памяти
N = 1	Массив неизменен; допустим как дополнительный крайний тест
N = 2, 4, 8, 16	Полное совпадение со std::sort, возможность ручной трассировки
Все элементы одинаковые	Значения сохраняются и результат отсортирован
Есть 0 и UINT32_MAX	Отсутствуют ошибки знакового сравнения и переполнения
Много одинаковых ключей	Корректное число каждого значения
Обратный порядок	Итог по возрастанию
N не степень двойки	Отказ или явно документированное дополнение входного массива
CPU vs GPU	Побитовое совпадение всех отсортированных uint32_t

Контрольная проверка: массив отсортирован по возрастанию и полностью совпадает с эталоном по значениям. Недостаточно проверять только сумму — она не выявляет все ошибки. При многообразии входных распределений разные ошибки конкурирующих потоков обнаруживаются гораздо легче.

8. Что включить в ~~отчёт~~ голову

1. Краткое объяснение битонной сети, её сложности и назначения стадий (k, j).
2. Модель CPU, количество физических ядер и логических потоков, RAM; модель GPU и VRAM; версии инструментов и ключевые параметры сборки.
3. Описание входных наборов, seed, используемых N и границ измерения времени.
4. Таблицу результатов корректности, исходный CSV, не менее пяти типов графиков.
5. Сравнение bitonic serial ↔ OpenMP ↔ CUDA (одинаковый алгоритм) и отдельное сравнение со std::sort (практический ориентир).
6. Обоснованный вывод: где выгодна параллельность, где её цена превышает выигрыш, какой вариант рекомендовать для выбранных N и данных.

Не допускается делать вывод «GPU быстрее CPU» только по одному прогону или исключительно по T_kernel. Результаты, в которых GPU медленнее std::sort, являются нормальными и могут быть особенно показательными при правильном объяснении.

9. Вопросы для повышения самооценки

1. Чем сортировка сравнением отличается от сортировки по разрядам (radix sort)?
2. Что такое сортировочная сеть и почему порядок её сравнений не зависит от исходных значений?
3. Какая последовательность называется битонной?
4. Почему базовая битонная сортировка обычно требует N = 2^m?
5. Как работают параметры k и j и операция XOR при определении пары?
6. Почему на одной стадии достаточно обрабатывать только i < (i XOR j)?
7. Сколько стадий у сети для N = 2^m и какова её асимптотическая сложность?
8. Почему между стадиями необходимо завершить все обмены предыдущей?
9. Чем барьер OpenMP отличается от CUDA __syncthreads()?
10. Почему __syncthreads() недостаточен для синхронизации блоков одной CUDA-сетки?
11. Почему последовательные запуски CUDA-ядер в одном stream подходят для базовой реализации?
12. Что из измеряемого времени входит в T_kernel, T_transfer и T_total?
13. Почему сравнение с std::sort не равно оценке ускорения именно битонного алгоритма?
14. Как входное распределение может влиять на std::sort и почему битонная сеть менее адаптивна?
15. Что произойдёт, если CUDA-сортировка будет запускаться без проверки ошибок?

16. Почему при увеличении количества OpenMP-потоков возможно замедление?
17. Что означает пропускная способность в элементах/с и чем она отличается от скорости передачи памяти?
18. Какие реальные задачи обработки данных выигрывают от сортировки и в каких случаях стоимость её выполнения может быть неоправданной?

10. Критерии принятия базовой лабораторной

Критерий	Проверка
Алгоритм	Корректная сеть, самостоятельная реализация compare-and-swap
CPU/OpenMP	Sequential и OpenMP, правильная синхронизация стадий
CUDA	Корректная GPU-версия, проверка CUDA-ошибок
Тестирование	Граничные и обычные данные; сравнение с std::sort
Эксперименты	Повторы, CSV, графики, T_kernel/T_total, CPU speedup
Защита	Интерпретация ускорения и замедления, практические выводы

Лабораторная считается выполненной после успешной защиты всех обязательных элементов. Высокое ускорение само по себе не требуется: корректность, воспроизводимость и понимание причин результата важнее численного рекорда.

11. Дополнительные исследования (для ботаников)

- Реализовать дополнение до ближайшей степени двойки с маркером UINT32_MAX, корректно учитывая реальные UINT32_MAX во входных данных.
- Сравнить повторное создание OpenMP parallel for на каждой стадии с одним долгоживущим parallel-регионом и необходимыми барьерами.
- Для малых битонных подпоследовательностей реализовать CUDA shared-memory вариант; объяснить границы синхронизации внутри блока.
- Сравнить свою GPU-сортировку с алгоритмом radix sort из библиотеки NVIDIA CUB при наличии совместимой версии библиотеки.
- Добавить сортировку пар (ключ, идентификатор), исследовать устойчивость (stability) и объяснить, почему стандартная битонная сортировка не обязана сохранять порядок равных ключей.

12. Документация и источники

- Microsoft Learn. /openmp (Enable OpenMP Support): https://learn.microsoft.com/en-us/cpp/build/reference/openmp-enable-openmp-2-0-support
- OpenMP Architecture Review Board. Specifications: https://www.openmp.org/specifications/
- NVIDIA. CUDA C++ Programming Guide (архив CUDA 12.9.1): https://docs.nvidia.com/cuda/archive/12.9.1/cuda-c-programming-guide/index.html
- NVIDIA. CUDA Runtime API (Events): https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__EVENT.html
- C++ reference: std::sort: https://en.cppreference.com/w/cpp/algorithm/sort.html
- NVIDIA CUB — Device-Wide Parallel Algorithms: https://nvidia.github.io/cccl/cub/api/structcub_1_1DeviceRadixSort.html
- Python Matplotlib documentation: https://matplotlib.org/stable/

Методическое замечание: данная работа специально использует один понятный сортировочный алгоритм для трёх архитектур исполнения. В реальных приложениях выбор алгоритма должен основываться на характере данных, общих затратах, объёме памяти и доступных оптимизированных библиотеках.